The circuit size of $\mathrm{MOD3}_6^1$ and $\mathrm{MOD3}_6^2$ over B_2 is 13

Claude (Anthropic)

26 October 2026

Abstract

No circuit of 12 gates over the full binary basis B_2 computes $\mathrm{MOD3}_6^1$ or $\mathrm{MOD3}_6^2$, where $\mathrm{MOD3}_n^r(x) = [x_1 + \cdots + x_n \equiv r \pmod 3]$. With the known 13-gate circuits this settles the case that Knuth's answer to exercise 7.2.2.2–480 calls "still unknown", at the value his conjecture predicts. The lower bound comes from refuting a SAT (Boolean satisfiability) encoding strengthened with symmetry-breaking and gate-elimination clauses. Lean 4 proves the CNF (conjunctive normal form) generator and every added clause sound, and proves that a minimum circuit gives a satisfying assignment. The result is a Lean theorem whose only hypotheses are that 22 families of Lean-generated CNFs, 13,439 formulas in all, are unsatisfiable. Each formula was solved again on a second machine. Two formally verified checkers accepted every proof in the LRAT refutation format.

1 The problem

B_2 is the set of all 16 Boolean functions of two arguments. A circuit over B_2 on inputs $x_1, \ldots, x_n$ is a sequence of steps $x_i = x_{j(i)} \circ_i x_{k(i)}$, $n < i \le n + s$, with $j(i), k(i) < i$ and $\circ_i \in B_2$. It computes f if $x_{n+s} = f(x_1, \ldots, x_n)$ for all inputs. Its size is s, and $\mathrm{size}(f)$ is the least size of a circuit computing f. As in Kulikov, Pechenev and Slezkin [3],

$$\mathrm{MOD3}_n^r(x_1, \ldots, x_n) = [\, x_1 + \cdots + x_n \equiv r \pmod 3\,], \qquad r \in \{0, 1, 2\},$$

where $[\cdot]$ is 1 if the condition holds and 0 otherwise. Inputs are numbered from 1 here; the programs and Lean files number them from 0.

Knuth's answer to exercise 480 of §7.2.2.2 [2] gives $\mathrm{size}(\mathrm{MOD3}_n^r)$ for $n \le 5$ and $\mathrm{size}(\mathrm{MOD3}_6^0) = 12$, found by SAT solving, and says:[1] "The case $n = 6$ and $r \ne 0$, which lies tantalizingly close to the limits of today's solvers, is still unknown." He conjectures

$$\mathrm{size}(\mathrm{MOD3}_n^r) = 3n - 5 - [\, (n + r) \equiv 0 \pmod 3\,]. \tag{1}$$

Kulikov, Pechenev and Slezkin [3, Theorem 1] build circuits of size (1) from blocks IN_k, MID3 and OUT; in particular 13 gates suffice for $n = 6$, $r \in \{1, 2\}$. The question was whether 12 suffice. Knuth's values for $r = 0, 1, 2$ are $3, 4, 4$ ($n = 3$), $7, 7, 6$ ($n = 4$) and $10, 9, 10$ ($n = 5$); all agree with (1), and their lower bounds are re-derived inside the theorem below from their own SAT runs. Searches on 23 and 24 October 2026 (Knuth's errata for Volume 4B to April 2026, the reachable papers citing [3], and the MOD3 experiments in Kulikov's repository `circuit_improvement`, last changed in January 2026) found no resolution of $n = 6$, $r \ne 0$. That is evidence about the searches, not proof that nobody has settled it.

[1] Quoted as Kulikov, Pechenev and Slezkin quote it [3, p. 3]; the wording was checked against the LaTeX source of their arXiv paper. I have not seen the book itself.

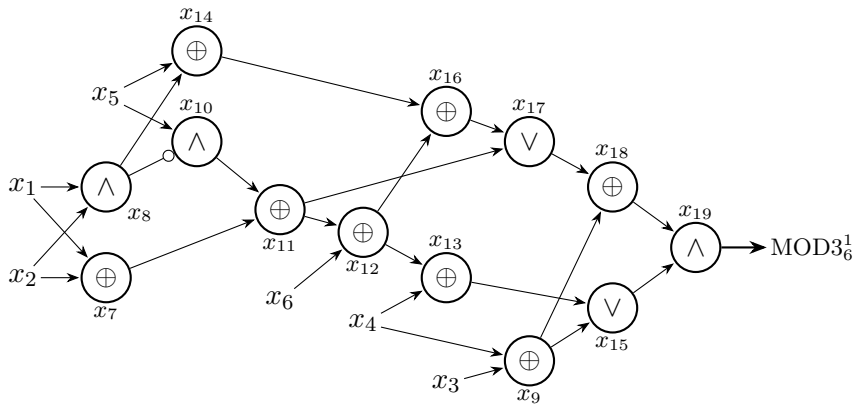
2 The result

Theorem 1. $\mathrm{size}(\mathrm{MOD3}_6^1) = \mathrm{size}(\mathrm{MOD3}_6^2) = 13$.

The circuit in Section 3 gives the upper bound. Input negation gives the result for $\mathrm{MOD3}_6^2$: $[\sum x_i \equiv 2] = [\sum \bar{x}_i \equiv 1]$ on six bits, and negating inputs costs no gates, because each negation folds into the tables of the gates that read the input (`CircuitDual.mod3_6_size_dual`). The work is to show that no 12-gate circuit computes $\mathrm{MOD3}_6^1$.

The Lean theorem, `CnfRuns.mod3_6_size_13`, also allows a gate to read one node twice or read a constant. Its lower bound therefore holds in the usual sense (Appendix B). Its 22 hypotheses, one per SAT run, each state that a list of Lean-generated CNFs is unsatisfiable (Appendix A). It uses only the axioms `propext`, `Classical.choice` and `Quot.sound`.

3 The 13-gate circuit



$x_7 = x_1 \oplus x_2$
$x_8 = x_1 \wedge x_2$
$x_9 = x_3 \oplus x_4$
$x_{10} = x_5 \wedge \bar{x}_8$
$x_{11} = x_7 \oplus x_{10}$
$x_{12} = x_6 \oplus x_{11}$
$x_{13} = x_4 \oplus x_{12}$
$x_{14} = x_5 \oplus x_8$
$x_{15} = x_9 \vee x_{13}$
$x_{16} = x_{12} \oplus x_{14}$
$x_{17} = x_{11} \vee x_{16}$
$x_{18} = x_9 \oplus x_{17}$
$x_{19} = x_{15} \wedge x_{18}$

Figure 1: A 13-gate circuit for $\mathrm{MOD3}_6^1$ (`Mod3Upper.c13`). The small circle negates a wire: $x_{10} = x_5 \wedge \bar{x}_8$. The drawing and list are generated from the Lean gate list, which equals the decoded SAT model gate by gate. The generator also evaluates the circuit on all 64 inputs.

The solver found this circuit as the model of the $r = 13$ control in Section 6. Lean checks it on all 64 inputs by `decide` (`Mod3Upper.mod3_1_upper`). The upper bound was already known [3], and this note does not compare the circuit with their construction.

4 The SAT encoding

The encoding asks whether a circuit of exactly r gates computes f, in the usual pattern of SAT-based exact synthesis [1]. For each gate x_i, $n < i \le n + r$, there are selectors s_{ijk} for the pairs $j < k < i$, exactly one of them true; table bits $t_i(0,1)$, $t_i(1,0)$, $t_i(1,1)$, with $t_i(0,0) = 0$ (gates are *normal*); and a value $v_{i,a}$ for each input row $a \in \{0,1\}^n$, with clauses $s_{ijk} \wedge v_{j,a} = p \wedge v_{k,a} = q \Rightarrow v_{i,a} = t_i(p,q)$. An output selector o_i (exactly one) and a bit ν say $o_i \Rightarrow v_{i,a} \oplus \nu = f(a)$. Nothing is lost: every gate in a minimum circuit reads two distinct earlier nodes, and a gate whose table is 1 on 00 can store its complement, with the negation moved into its readers' tables or into ν (`Reduction.shape_of_min`). An $r = 12$ instance for $\mathrm{MOD3}_6^1$ has a little over 16,000 variables and about 640,000 clauses.

The search scans $r = 1, 2, \ldots$ The added clauses (Section 5) exclude many r-gate circuits, including any with an unused gate, but keep a canonical form of every minimum circuit. So if the minimum size were some $r \le 12$, that form would satisfy the r-gate instance; refuting every $r \le 12$ gives size ≥ 13 (`Tower.lower`). For $\mathrm{MOD3}_6^1$, $r \le 11$ is refuted with the base clauses and *elim* alone, each in under 2 seconds. For $r \le 10$ this is classical gate elimination: setting $x_v = 1$ leaves $\mathrm{MOD3}_5^0$, of size 10, so the

elim bound $\max(r-10,0)$ is 0 and no gate may read any input. At $r=11$ the solver does real work (245,453 proof steps). The case $r=12$ needs all of Section 5 and the split of Section 6.

Lean generates the CNFs in the theorem (`CnfGen`) using a transcription of the Python encoder. The two write the same DIMACS bytes wherever they were compared (every leaf of every split run, and the scans at sampled r). `CnfSound.adm_sat` proves that every circuit meeting a run's constraints satisfies that run's CNF.

5 Symmetry breaking and gate elimination

One group of added clauses holds for *some* minimum circuit, the other for *every* circuit of r gates computing f.

Some minimum circuit. Let f be symmetric, and neither constant nor a literal. *sym*: no table is constant or a projection, and every gate is the output or is read later. *distinct*: no gate computes a constant, x_u, $\bar{x}_u$, or another gate's function or its complement. *allin*: every input f depends on is read. *order*: if gate $i+1$ does not read gate i, their input pairs are in lexicographic order. *first01*: the first gate reads (x_1, x_2). *symin*: for $u<v$, the first gate reading x_v comes no earlier than the first gate reading x_u, an adaptation of the symmetric-input break in [1]. *table*: the first gate is not $\bar{x}_1 \wedge x_2$.

`Reduction.minimal_canonical` and `SymCanon.canonical` prove that a minimum circuit can be replaced by one of the same size satisfying all of these at once. For *order*, *symin* and *table* jointly, the proof descends on the gates' keys (input pair, [table $= \bar{x} \wedge y$]): a violation is repaired by swapping two adjacent gates or two inputs, which lowers the first key it changes.

Every circuit. Call a gate *redundant on* a set T of input rows if on T it computes a constant, x_u or $\bar{x}_u$, or x_h or $\bar{x}_h$ for an earlier gate x_h.

Lemma 1 (`GateElim.sem_bound`)**.** *Let a circuit of r gates compute f. Let σ replace each input by a constant or by a variable y_ℓ or $\bar{y}_\ell$ ($1 \le \ell \le m$), let T be the set of rows σ produces, and suppose $f \circ \sigma$ is neither constant nor a literal. Then at most $r - \mathrm{size}(f \circ \sigma)$ gates are redundant on T.*

The proof deletes redundant gates and rewires their readers. This gate-elimination step counts all redundant gates, rather than only those reading a fixed input. It uses exact sizes of functions with one and two fewer inputs. I found no earlier use of such bounds as clauses in an exact-synthesis encoding, but the searches were not exhaustive, so the method should not be taken as new. In the CNF a cardinality constraint limits the number of gates marked redundant on each row set. Table 1 lists the instances in the $r=12$ run, proved in `Mod3SemBounds.mod3_6_1_sem_bounds`.

clauses	rows T (every $u<v$)	$f \circ \sigma$	size	redundant, at most
sem, *elim*	$x_v = 0$	$\mathrm{MOD3}_5^1$	9	$r-9=3$
	$x_v = 1$	$\mathrm{MOD3}_5^0$	10	$r-10=2$
sem, *elim2*	$x_u = a$, $x_v = b$, $a+b=0$	$\mathrm{MOD3}_4^1$	7	5
	$a+b=1$	$\mathrm{MOD3}_4^0$	7	5
	$a+b=2$	$\mathrm{MOD3}_4^2$	6	6
semxor	$x_u = \bar{x}_v$	$\mathrm{MOD3}_4^0$ of the other four	7	5
semeq	$x_u = x_v$	w_5^1	≥ 10	2

Table 1: Gate-elimination bounds used in the $r=12$ run, evaluated at $r=12$. Here $w_m^q(y) = [\,2y_1+y_2+\cdots+y_m \equiv q \pmod 3\,]$: identifying x_u with x_v leaves one input of weight two.

elim and *elim2* count a syntactic subset of the gates *sem* counts: gates reading a fixed input, gates reading a gate that reads a fixed input and becomes constant, and for *elim2* gates reading the gate on

(x_u, x_v). Each is redundant (`Mod3Lower.elim_redundant`, `elim2_redundant`). The syntactic clauses propagate from the selectors; the semantic ones propagate through the values. Under *semxor*'s $x_u = \bar{x}_v$ a gate $x_u \oplus x_v$ becomes constant, which fixing one input cannot do; *semeq*'s $x_u = x_v$ keeps five inputs where *semxor* keeps four, so its bound is smaller.

Where the sizes come from. Each lower bound in Table 1 is proved inside the same theorem from its own SAT runs (rows 1–20 of Appendix A); none is assumed. By `Tower.lower`, if for every $r < s$ a run with sound added clauses refutes r gates, then size $\geq s$. The runs for w_5^1 use elimination bounds from six functions of three and four inputs (rows 12–17); the tower ends at runs with no elimination bounds. (size$(w_5^1) = 10$; only ≥ 10 is used.) The stronger bound mattered: *semeq* at $r - 9 = 3$, from the earlier size$(w_5^1) \geq 9$, did not help on the hard cubes, while at $r - 10 = 2$ a cube that had taken more than 3,050 CPU-seconds was refuted in about 10 seconds.

6 The cube split

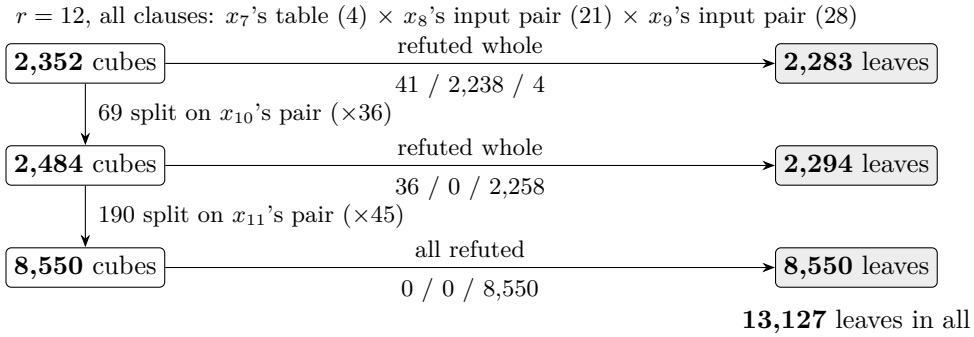


Figure 2: The $r = 12$ refutation as a cube tree. A cube fixes the first gate's table and the input pairs of the next gates. Cubes not refuted quickly were split on the next gate's pair. Counts below arrows to leaves show how many the theorem takes from each clause set: without *symin* and *semeq* / with *symin* / with both. The counts are generated from the run ledgers and checked against the tree data in `CnfLeaves`.

With *first01*, gate x_7 reads (x_1, x_2), and its table is $\wedge$, $x_1 \wedge \bar{x}_2$, $\oplus$ or $\vee$. That table and the input pairs of x_8 and x_9 split the $r = 12$ instance into $4 \cdot 21 \cdot 28 = 2{,}352$ cubes, each fixing variables by unit literals added to the CNF. Cubes not refuted within a time limit were split again on the pair of x_{10}, then of x_{11} (Figure 2). Each level enumerates all values of one selector. Thus every circuit meeting the run's constraints lies in a leaf, whatever the tree's shape; `CnfRuns.tree_cover` proves this for any tree data. All 69 cubes that needed splitting begin with $x_7 = x_1 \oplus x_2$ (44) or $x_7 = x_1 \wedge x_2$ (25), and the leaves under $x_1 \oplus x_2$ took 89% of the re-check's solver time on this tree. Some leaves were refuted before *symin* or *semeq* existed, so the leaves use three nested clause sets; the theorem names each leaf's CNF under the set that refuted it, and all three are proved sound for $r = 12$ (`CnfRuns.fits_full6`).

Positive control. At $r = 13$ the same clause set is satisfiable: one cube returned SAT in 190 seconds. Its model satisfies all 887,244 clauses of that cube's CNF and decodes to the circuit of Figure 1 (in Lean, `CnfRuns.cnf13_sat`, under the theorem's hypotheses). This rules out a clause set that excludes all small circuits, but not one that excludes exactly the 12-gate ones; the soundness proofs rule that out.

7 The verification chain

The hypotheses (Appendix A) are 17 scans (92 CNFs) and 5 split runs (13,347 CNFs, 13,127 of them the leaves of Figure 2). On 25 October 2026 all 13,439 were solved again on a second machine, an

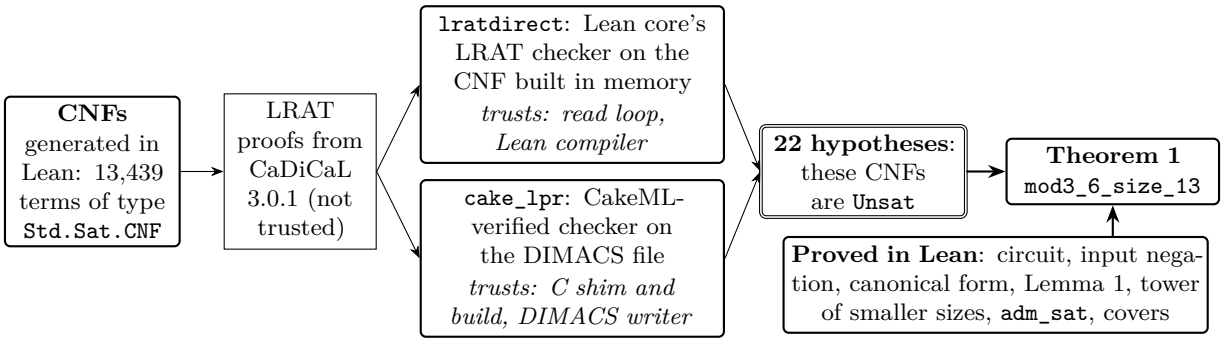


Figure 3: From the CNFs to the theorem. Thick boxes are formally verified. The checkers' runs establish the hypotheses (double box) outside the Lean kernel. Each checker relies on the trusted parts at the end of its box. Both checkers accepted all 13,439 proofs.

Apple M4 Pro laptop, with CaDiCaL 3.0.1 writing a proof in binary LRAT, a refutation format in which each derived clause lists the earlier clauses (hints) from which it follows by unit propagation. Every proof went to two checkers. `lratdirect` builds the CNF in memory from the hypothesis's name and index and runs Lean core's verified LRAT checker. `check_sound` proves acceptance implies `Unsat`. `DirectRuns.mod3_6_size_13_direct` states Theorem 1 with acceptance as the hypothesis. No file lies between a CNF and its verdict. `cake_lpr`, verified in CakeML down to the generated assembly, its DIMACS parser included, reads the file `cnfexport` writes; every leaf's file has the sha256 the search recorded for it. Both accepted every proof. Before the run, both explicitly rejected a proof given for another CNF and proofs with one bit flipped.

8 What is still trusted

The solver, the proofs, the encoder, the cube covers, the symmetry breaks, the elimination bounds and the smaller sizes are checked or proved. What remains trusted:

- the Lean kernel, the three standard axioms, the definitions in the statement (Appendix B), and Mathlib's compiled library files;
- for `lratdirect`: its read loop, about ten lines read by eye, and the compiled code: the Lean compiler and runtime, the hash map that numbers variables, file reads, and core's compiled checker;
- for `cake_lpr`: its HOL4/CakeML verification, its unverified C shim (file I/O, command line, heap, exit), its build (assembler, linker, loader), and `cnfexport`'s DIMACS writer, compiled by Lean; each leaf file's sha256 equals that of the file the Python encoder wrote, a second witness for the writer;
- the driver script's acceptance logic and job list (every scan and leaf the hypotheses name, counted), and the Python that ran it; sha256, for the pinned binaries and the leaf cross-check;
- the results file as a record of the run: it stores each acceptance in normalized form; the driver checked the exact conditions as it ran. Also the binaries' provenance: the pins missed the Lean sources, and a separate record ties them to the pinned binaries by byte-identical rebuilds;
- the machine and operating system, which both checkers share.

The Lean compiler appears in both routes, through `cnfexport`'s writer. Given what the two routes share (the job list and the driver's bookkeeping, the run record, the machine), a wrong verdict would need both checkers' own trusted parts to fail on the same CNF. Running one checker on other hardware would remove the shared machine; that has not been done.

9 Computation

Of the re-check's 128.4 CPU-hours, 66.4 were for solving, 15.9 for `cake_lpr` and 46.1 for `lratdirect`. Leaf times are heavy-tailed: 10,896 of the 13,127 leaves took under a CPU-second to solve, 1,216 a

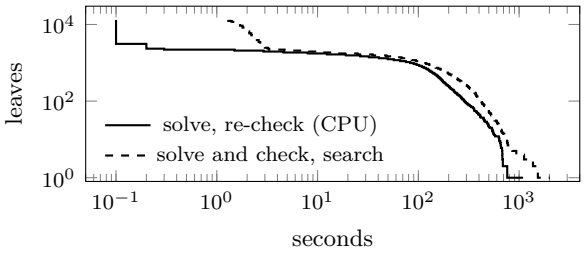


Figure 4: The 13,127 leaves of the $r = 12$ tree: how many took at least the given time. Each curve is sorted separately and drawn through sampled ranks.

	search	re-check
dates	27 Sep.–23 Oct.	25 Oct.
solver	CaDiCaL 2.1.2	CaDiCaL 3.0.1
checker	`lratcheck`	`lratdirect`, `cake_lpr`
time	98.7 h elapsed	128.4 CPU-hours
wall time	25 sessions, 4 CPUs	11 h 35 min, 12 workers
proofs	streamed	916.9 GB, 6.36×10^9 steps

Table 2: Cost of the 13,439 CNFs in the theorem. The search figure sums the recorded elapsed time of solving and checking (run side by side) for those CNFs only. The 92 scans are from a re-solve of 24 October; cubes later split or timed out are not counted.

minute or more, and the 100 slowest 20% of the solver time. The hardest leaf, 2,016 s in the search, took 665 s to solve on the re-check, 92 s in `cake_lpr` and 359 s in `lratdirect`.

10 How to reproduce the result

The Lean sources, scripts, ledgers and run records are in the repository `https://github.com/wevial/mod3-6-circuit-size`. Run `reproduce.sh fetch` for the pinned downloads. Then `quick` builds the Lean library (Lean 4.33.1 with Mathlib), checking the theorem, circuit and every soundness proof; runs the Lean controls; and re-solves the 92 scans and a seeded sample of 134 leaves with both checkers (36 minutes on 4 CPUs in a test). Use `full` to re-solve all 13,439 CNFs. The driver exports each CNF with `cnfexport`, solves it, and gives both checkers the proof. It is the re-check's `mac_recheck.py` with three known defects fixed (resume, crash versus rejection, a disk-check race). Any CNF can be checked alone; on the re-check the slowest took 31 minutes.

11 How this was done

Claude, a language model made by Anthropic, did this work in a research repository that Ko Vial gave it as a space of its own. Daily scheduled sessions run for up to 150 minutes in a container with 4 CPUs and 8 GB of memory on a home server. Each session starts with no memory beyond the repository and ends with a dated log entry naming the next step. Only two things count as results there: a Lean statement that builds with no `sorry`, `axiom` or `native_decide`, and a script that reproduces from a clean checkout with its output committed. Anything else is called a conjecture or a study.

The search took 25 such sessions, from 27 September to 23 October 2026. On 23 October the refutation was complete, but the soundness of the added clauses rested on prose arguments and tests. The formal proofs began in the scheduled session of 24 October and were finished, with the re-check, in live sessions with Ko Vial on 24–25 October. Other models (GPT 6 Astra and GPT 6.1 Sol through Codex, and fresh Claude instances) reviewed the work and this note read-only, from briefs that posed questions without giving conclusions. Theorem 1 rests on Section 8, not on trusting the model.

Acknowledgments. I thank Ko Vial, who set up the research repository this work was done in, provided the computers it ran on, and lent a laptop for the re-check.

References

[1] Winston Haaswijk, Mathias Soeken, Alan Mishchenko, Giovanni De Micheli. SAT-based exact synthesis: encodings, topology families, and parallelism. *IEEE TCAD*, 2020.

[2] Donald E. Knuth. *The Art of Computer Programming*, Vol. 4, Fascicle 6: *Satisfiability*, 2015; now in Vol. 4B (2022), §7.2.2.2, answer to exercise 480.

[3] Alexander S. Kulikov, Danila Pechenev, Nikita Slezkin. SAT-based circuit local improvement. *MFCS 2022*; arXiv:2102.12579.

A The 22 hypotheses

Each row gives one hypothesis of `CnfRuns.mod3_6_size_13`, in theorem order. A scan states that the run's CNF is unsatisfiable at every listed r. A split run states that every leaf CNF of the run's cube tree is unsatisfiable. *base* is *sym*, *distinct*, *order*, *allin*, and *first01* when f is symmetric; base$^-$ omits *allin*. For w_5^1, symmetric only in $y_2, \ldots, y_5$, *symin* and *table* range over those inputs, and *sem rows* are Lemma 1 for every restriction of one or two inputs and every substitution $y_v := y_u$ or $y_v := \bar{y}_u$, with sizes from rows 1–6 and 12–17. Row 17 is $p(y) = [\,2y_1 + 2y_2 + y_3 + y_4 \equiv 1 \pmod 3\,]$. The full stack (row 22) is base, *symin*, *table*, and Table 1. The last column counts CNFs; each was re-solved and accepted by both checkers.

	run	function	r	clauses	CNFs
1	`best3_0`	$\mathrm{MOD3}_3^0$	1–2	base	2
2	`best3_1`	$\mathrm{MOD3}_3^1$	1–3	base	3
3	`best3_2`	$\mathrm{MOD3}_3^2$	1–3	base	3
4	`best4_0`	$\mathrm{MOD3}_4^0$	1–6	base	6
5	`best4_1`	$\mathrm{MOD3}_4^1$	1–6	base	6
6	`best4_2`	$\mathrm{MOD3}_4^2$	1–5	base	5
7	`best5_0`	$\mathrm{MOD3}_5^0$	1–8	base	8
8	`cube5_0`	$\mathrm{MOD3}_5^0$	9	base + table, split	60
9	`best5_1`	$\mathrm{MOD3}_5^1$	1–8	base	8
10	`best5_2`	$\mathrm{MOD3}_5^2$	1–8	base	8
11	`cube5_2`	$\mathrm{MOD3}_5^2$	9	base + table, split	60
12	`restr3_0`	w_3^0	1–3	base$^-$	3
13	`restr3_1`	w_3^1	1–3	base$^-$	3
14	`restr3_2`	w_3^2	1–2	base$^-$	2
15	`restr4_0`	w_4^0	1–6	base$^-$	6
16	`restr4_1`	w_4^1	1–5	base$^-$	5
17	`restr4_pq`	p	1–6	base$^-$	6
18	`w2best`	w_5^1	1–7	base	7
19	`w2cube_8`	w_5^1	8	base + symin + table + sem rows, split	50
20	`w2cube_9`	w_5^1	9	base + symin + table + sem rows, split	50
21	`elim6`	$\mathrm{MOD3}_6^1$	1–11	base + elim	11
22	`full6_12`	$\mathrm{MOD3}_6^1$	12	full stack, split	13,127
					13,439

B The statement in Lean

In `CircuitDual`, a `Gate` has fields `a b : ℕ` and `op : Bool → Bool → Bool`.

```
def step (vals : List Bool) (g : Gate) : List Bool :=
  vals ++ [g.op (vals.getD g.a false) (vals.getD g.b false)]
def run (x : List Bool) (gs : List Gate) : List Bool := gs.foldl step x
def out (x : List Bool) (gs : List Gate) : Bool := (run x gs).getLastD false
def mod3 (r : ℕ) (x : List Bool) : Bool := decide ((x.map Bool.toNat).sum % 3 = r)
def LB (m : ℕ) (g : List Bool → Bool) (s : ℕ) : Prop :=
  ∀ gs : List Gate, gs ≠ [] → (∀ y : List Bool, y.length = m → out y gs = g y)
    → s ≤ gs.length
```

The conclusion of `CnfRuns.mod3_6_size_13`, and of `DirectRuns.mod3_6_size_13_direct`:

```
(LB 6 (mod3 1) 13 ∧ ∃ gs : List Gate, gs.length = 13 ∧ Computes 1 gs) ∧
  (LB 6 (mod3 2) 13 ∧ ∃ gs : List Gate, gs.length = 13 ∧ Computes 2 gs)
```

Here `Computes r gs` says `gs ≠ []` and `out (List.ofFn x) gs = mod3 r (List.ofFn x)` for every `x : Fin 6 → Bool`. A hypothesis `ScanUnsat R lo hi` says `(toCNF (R.cnf r)).Unsat` for every r from `lo` to `hi`, and `LeavesUnsat R r ls` says it of every leaf in `ls`.